# Atelier 3 : Express, l'API GreenCart

GreenCart est une petite application web de vente de produits locaux. Dans cet atelier, vous construisez son API avec Express : la liste des produits, l'ajout, la modification, la suppression, et les middlewares qui la rendent propre.

Rien de nouveau ici. Vous reprenez le cours Node.js et Express, et vous l'appliquez sur un projet complet. Le projet est déjà organisé comme dans le cours. Deux routes sont écrites pour vous servir d'exemple.

Les exercices sont classés par priorité. Faites-les dans l'ordre.

| Exercice | Priorité | Sujet |
|---|---|---|
| 1 | Indispensable | Filtrer les produits par catégorie |
| 2 | Indispensable | Ajouter un produit (POST, 201) |
| 3 | Indispensable | Modifier et supprimer un produit (PUT, DELETE, 404, 204) |
| 4 | Souhaitable | Les middlewares : journal, validation (400), erreurs |
| 5 | Bonus | Une page d'accueil avec le module `http`, sans Express |

> **Les numéros de slides** sont ceux du diaporama du cours Node.js et Express. Dans la version PDF, la slide 12 est masquée : à partir de la slide 13, la page du PDF vaut le numéro de slide moins 1. La slide 22 est la page 21 du PDF.

## Avant de commencer

Il vous faut Node 18 ou plus. Vérifiez avec `node -v`.

Ouvrez un terminal dans ce dossier et installez les dépendances :

```
npm install
```

Cette commande lit `package.json` et installe Express, et nodemon pour le développement (cours, slides 10 et 15). Le dossier `node_modules` apparaît.

### Lancer le serveur

```
npm run dev
```

`npm run dev` lance `nodemon server.js` : le serveur redémarre tout seul à chaque fois que vous enregistrez un fichier. Vous devez voir :

```
Serveur GreenCart lancé sur http://localhost:3000
```

Sans nodemon, `npm start` lance `node server.js`. Il faut alors l'arrêter (Ctrl + C) et le relancer après chaque modification.

Ouvrez http://localhost:3000/api/produits dans le navigateur : la liste des produits s'affiche en JSON.

Si le port 3000 est déjà pris, choisissez-en un autre. Sur Mac et Linux : `PORT=3002 npm run dev`. Sous Windows, remplacez 3000 par 3002 dans `server.js`.

### Vérifier votre travail

Laissez le serveur tourner. Ouvrez un **deuxième terminal**, dans le même dossier, et lancez :

```
node verifier.js
```

Le vérificateur appelle votre API et affiche une ligne par exercice, `OK` ou `À revoir` avec une explication. Lisez l'explication, corrigez, relancez. Il crée ses propres produits de test et les supprime à la fin. Vous pouvez le relancer autant de fois que vous voulez.

Si votre serveur tourne sur un autre port, donnez-le : `node verifier.js 3002`.

Au départ, seule la ligne « Point de départ » est OK. C'est normal.

### Tester une route à la main

Le navigateur ne sait envoyer que des requêtes GET. Pour POST, PUT et DELETE, utilisez Postman, ou l'extension Thunder Client de VS Code. Dans les exemples ci-dessous, une requête s'écrit ainsi :

```
POST /api/produits
Content-Type: application/json

{ "nom": "Courgettes", ... }
```

Cela veut dire : méthode POST, adresse http://localhost:3000/api/produits, et ce JSON dans le corps de la requête (onglet Body, format JSON).

## Le projet

Il suit la structure du cours (slide 22) :

```
etudiant/
├── server.js                       le point d'entrée, il démarre le serveur
├── app.js                          configure Express et l'exporte
├── routes/produit.routes.js        les chemins de l'API des produits
├── controllers/produit.controller.js la logique de chaque route
├── models/produit.model.js         les données (fourni, ne pas modifier)
├── middlewares/                    journal, validation, erreurs (exercice 4)
├── data/                           produits.json et producteurs.json
├── accueil-http.js                 le bonus (exercice 5)
└── verifier.js                     le vérificateur (ne pas modifier)
```

Le lancement suit les étapes du cours (slide 14) : `app.js` crée l'application et l'exporte avec `module.exports = app`, `server.js` la récupère avec `require('./app')` et la fait écouter sur un port.

Une requête suit le chemin de la slide 40 : la route reçoit la requête, appelle une fonction du contrôleur, qui demande les données au modèle, puis répond en JSON.

### Le modèle

Il n'y a pas encore de base de données. Le modèle lit `data/produits.json` au démarrage et garde les produits en mémoire, dans un tableau. Chaque redémarrage du serveur remet le catalogue d'origine. L'atelier 4 branchera une vraie base.

Le modèle vous donne cinq fonctions :

| Fonction | Ce qu'elle fait | Ce qu'elle renvoie |
|---|---|---|
| `Produit.trouverTous()` | tous les produits | un tableau |
| `Produit.trouverParId(id)` | le produit qui a cet id | le produit, ou `null` |
| `Produit.creer(donnees)` | ajoute un produit, lui donne un id | le produit créé |
| `Produit.remplacer(id, donnees)` | remplace tout le produit | le produit modifié, ou `null` |
| `Produit.supprimer(id)` | supprime le produit | `true`, ou `false` s'il n'existait pas |

Attention : `id` doit être un **nombre**. Ce qui arrive dans `req.params.id` est une chaîne. L'exemple `afficherProduit` montre la conversion avec `Number()`.

### Un produit

```
{
  "id": 4,
  "nom": "Carottes",
  "categorie": "legumes",
  "prix": 1.8,
  "unite": "kg",
  "stock": 60,
  "bio": true,
  "producteurId": 1
}
```

Les catégories : `fruits`, `legumes`, `cremerie`, `epicerie`. Le prix est en euros, hors taxes.

### Les deux routes d'exemple

Elles sont déjà écrites. Lisez-les avant de commencer : tous les exercices s'en inspirent.

- `GET /api/produits` : la liste de tous les produits.
- `GET /api/produits/:id` : un produit. S'il n'existe pas, la réponse est `404` avec `{ "message": "Produit introuvable" }`.

Regardez comment elles sont écrites dans `routes/produit.routes.js` et dans `controllers/produit.controller.js`. Chaque fonction du contrôleur met son code dans un `try` et passe l'erreur à `next(err)` dans le `catch` (slide 31). Gardez cette forme.

---

## Exercice 1 : filtrer par catégorie

**Priorité : indispensable**

### Ce qui est demandé

Complétez `listerProduits` dans `controllers/produit.controller.js`. Quand l'adresse contient `?categorie=legumes`, la route ne renvoie que les légumes. Sans ce paramètre, elle renvoie tout, comme

maintenant. Une catégorie qui n'existe pas donne une liste vide.

### Rappel

- Cours Node.js : les routes, slides 23 et 24, la méthode GET, slide 19.
- Cours JavaScript : les conditions, slides 30 à 32, la boucle `for`, slide 34, `push` pour ajouter dans un tableau, slide 49.

### Résultat attendu

```
GET /api/produits?categorie=legumes
```

Réponse `200` :

```
[
  { "id": 4, "nom": "Carottes", "categorie": "legumes", ... },
  { "id": 5, "nom": "Poireaux", "categorie": "legumes", ... },
  { "id": 6, "nom": "Pommes de terre", "categorie": "legumes", ... },
  { "id": 7, "nom": "Radis", "categorie": "legumes", ... }
]
```

```
GET /api/produits?categorie=bonbons
```

Réponse `200` : `[]`

### Indice

Les paramètres après le `?` arrivent dans `req.query`. Ici : `req.query.categorie`. S'il est vide, renvoyez tout. Sinon, créez un tableau vide, parcourez les produits avec une boucle `for`, et ajoutez avec `push` ceux dont la catégorie est la bonne.

Testez dans le navigateur : http://localhost:3000/api/produits?categorie=fruits

### À rendre

`controllers/produit.controller.js`, avec `listerProduits` complétée.

---

## Exercice 2 : ajouter un produit

**Priorité : indispensable**

### Ce qui est demandé

Créez la route `POST /api/produits`. Elle reçoit un produit en JSON, l'ajoute au catalogue, et répond `201 Created` avec le produit créé, qui a maintenant un `id`.

Deux fichiers à modifier :

1. `routes/produit.routes.js` : ajoutez la route POST, sur le chemin `'/'`.
2. `controllers/produit.controller.js` : complétez `creerProduit`.

### Rappel

- Les méthodes HTTP, slide 19 : POST crée une ressource.
- Les codes de statut, slide 20 : les 2xx disent que tout s'est bien passé.
- Une route = une URL + une méthode HTTP + une action, slide 24.
- Une route par fichier métier, slide 25.

### Résultat attendu

```
POST /api/produits
Content-Type: application/json

{
  "nom": "Courgettes",
  "categorie": "legumes",
  "prix": 2.2,
  "unite": "kg",
  "stock": 25,
  "bio": true,
  "producteurId": 1
}
```

Réponse `201` :

```
{
  "id": 13,
  "nom": "Courgettes",
  "categorie": "legumes",
  "prix": 2.2,
  "unite": "kg",
  "stock": 25,
  "bio": true,
  "producteurId": 1
}
```

Ensuite, `GET /api/produits/13` renvoie bien les courgettes.

### Indice

Le produit envoyé est dans `req.body`. Donnez-le à `Produit.creer()`, qui renvoie le produit avec son id. Par défaut, Express répond `200`. Pour une création, le bon code est `201` : `res.status(201).json(...)`.

Dans le fichier des routes, prenez modèle sur la ligne `router.get('/', controleur.listerProduits)`.

### À rendre

`routes/produit.routes.js` et `controllers/produit.controller.js`.

---

## Exercice 3 : modifier et supprimer un produit

**Priorité : indispensable**

### Ce qui est demandé

Créez deux routes sur un produit précis, avec le chemin `'/:id'` :

- `PUT /api/produits/:id` remplace tout le produit. Réponse `200` avec le produit modifié. Si le produit n'existe pas : `404` avec `{ "message": "Produit introuvable" }`.
- `DELETE /api/produits/:id` supprime le produit. Réponse `204 No Content`, sans rien dans le corps. Si le produit n'existe pas : `404`, avec le même message.

Complétez `modifierProduit` et `supprimerProduit` dans le contrôleur.

### Rappel

- PUT remplace entièrement une ressource, DELETE la supprime, slide 19.
- Les codes de statut, slide 20 : les 4xx sont les erreurs du client.
- La route d'exemple `afficherProduit` fait déjà le 404.

### Résultat attendu

```
PUT /api/produits/4
Content-Type: application/json

{
  "nom": "Carottes",
  "categorie": "legumes",
  "prix": 1.95,
  "unite": "kg",
  "stock": 50,
  "bio": true,
  "producteurId": 1
}
```

Réponse `200` :

```
{ "id": 4, "nom": "Carottes", "categorie": "legumes", "prix": 1.95, "unite": "kg", "stock":
50, "bio": true, "producteurId": 1 }
```

```
DELETE /api/produits/4
```

Réponse `204`, sans contenu. Ensuite, `GET /api/produits/4` répond `404`.

```
DELETE /api/produits/999
```

Réponse `404` :

```
{ "message": "Produit introuvable" }
```

### Indice

Recopiez la forme de `afficherProduit` : convertir l'id avec `Number()`, appeler le modèle, tester le résultat, répondre.

- `Produit.remplacer(id, req.body)` renvoie `null` si le produit n'existe pas.
- `Produit.supprimer(id)` renvoie `false` s'il n'existe pas.
- Pour un 204 : `res.status(204).send()`.

Après vos tests, redémarrez le serveur pour retrouver le catalogue d'origine.

### À rendre

`routes/produit.routes.js` et `controllers/produit.controller.js`.

---

## Exercice 4 : les middlewares

**Priorité : souhaitable**

Un middleware est une fonction qui passe entre la réception de la requête et l'envoi de la réponse (slide 26). Il appelle `next()` pour passer la main au suivant. Sans `next()`, la requête reste bloquée (slide 27). Les trois fichiers sont prêts dans `middlewares/`, à compléter.

### 4.1 Le journal

**Ce qui est demandé**

Complétez `middlewares/journal.middleware.js` : il affiche dans le terminal la méthode et l'URL de chaque requête. Branchez-le dans `app.js` en middleware **global**, avant les routes.

**Rappel**

- L'exemple de la slide 27 est exactement ce journal.
- Global ou local, slide 29 : `app.use(logger)` s'applique à toutes les routes.

**Résultat attendu**

Quand vous ouvrez http://localhost:3000/api/produits/4, le terminal du serveur affiche :

```
Requête reçue : GET /api/produits/4
```

**Indice**

`req.method` et `req.url`. N'oubliez pas `next()` à la fin. Dans `app.js`, retirez les `//` devant la ligne `const journal = require(...)`, puis ajoutez `app.use(journal)` à l'endroit indiqué.

**À rendre**

`middlewares/journal.middleware.js` et `app.js`.

### 4.2 La validation

**Ce qui est demandé**

Complétez `middlewares/validation.middleware.js`. Il refuse un produit incorrect avec le code `400` et un message en JSON. Si tout est correct, il appelle `next()`. Les règles :

1. `nom` est une chaîne de caractères, non vide.
2. `categorie` vaut `fruits`, `legumes`, `cremerie` ou `epicerie`.
3. `prix` est un nombre plus grand que 0.
4. `stock` est un nombre entier, 0 ou plus.

Placez-le en middleware **local** sur les routes POST et PUT, avant la fonction du contrôleur.

#### Rappel

- Le middleware de validation, slide 28.
- Global ou local, slide 29 : `app.get("/users", authMiddleware, getUsers)`.
- Cours JavaScript : les conditions, slides 30 à 32, `typeof`, exercices de la slide 93, `indexOf()`, slides 44 et 45.

#### Résultat attendu

```
POST /api/produits
Content-Type: application/json

{ "nom": "Courgettes", "categorie": "legumes", "prix": "abc", "unite": "kg", "stock": 25 }
```

Réponse `400` :

```
{ "message": "Le prix doit être un nombre plus grand que 0" }
```

Le produit n'est pas créé. Un produit correct, lui, donne toujours `201`.

#### Indice

Une règle, un `if`. Dans chaque `if`, répondez tout de suite avec `return res.status(400).json({ message: '...' })`. Le `return` arrête la fonction : `next()` n'est pas appelé. Le premier `if` est donné en exemple dans le fichier.

- `typeof produit.prix !== 'number'` repère un prix envoyé en texte.
- `CATEGORIES.indexOf(produit.categorie) === -1` repère une catégorie inconnue.
- `Number.isInteger(produit.stock)` vaut `true` pour 25, `false` pour 2.5.

Dans le fichier des routes, le middleware se place entre le chemin et la fonction : `router.post('/', validerProduit, controleur.creerProduit)`.

#### À rendre

`middlewares/validation.middleware.js` et `routes/produit.routes.js`.

### 4.3 Les erreurs

#### Ce qui est demandé

Deux ajouts à la fin de `app.js`, dans cet ordre :

1. **Route introuvable.** Un middleware qui répond `404` avec `{ "message": "Route introuvable" }`. Express l'appelle quand aucune route n'a répondu.
2. **Le middleware d'erreur.** Complétez `middlewares/erreur.middleware.js` : il affiche `err.message` dans le terminal et répond `500` avec `{ "message": err.message }`. Branchez-le en tout dernier.

La route `GET /api/panne` est déjà dans `app.js`. Elle lance une erreur exprès, pour tester.

**Rappel**

- L'utilité de la gestion des erreurs, slide 30.
- `try/catch` et `next(err)`, slide 31 : c'est ce que font déjà les fonctions du contrôleur.
- Le middleware d'erreur, slide 32 : quatre paramètres, placé à la fin.

**Résultat attendu**

```
GET /api/une-route-inconnue
```

Réponse `404` : `{ "message": "Route introuvable" }`, et non plus la page HTML « Cannot GET ».

```
GET /api/panne
```

Réponse `500` :

```
{ "message": "Panne simulée pour tester le middleware d'erreur" }
```

**Indice**

- La route introuvable est un middleware ordinaire, sans `next` : `app.use((req, res) => { ... })`.
- Le middleware d'erreur a **quatre** paramètres : `(err, req, res, next)`. C'est à cela qu'Express le reconnaît. Gardez les quatre, même si `next` ne sert pas.
- L'ordre compte. Express lit `app.js` de haut en bas : routes, puis route introuvable, puis erreurs.

**À rendre**

`middlewares/erreur.middleware.js` et `app.js`.

---

## Exercice 5 : la page d'accueil, sans Express

**Priorité : bonus**

Express est construit au-dessus d'un module livré avec Node : `http`. Ce bonus vous montre ce qu'Express fait pour vous. Vous écrivez un petit serveur avec `http.createServer`, sans Express, sans routes, sans `res.json()`.

C'est la route « from scratch », conseillée aux étudiants Webecom : comprendre ce qu'il y a sous le framework. Ce module n'est pas dans le cours. Le squelette est fourni.

### Ce qui est demandé

Complétez `accueil-http.js`. Sur l'adresse `/`, il renvoie une page HTML avec le titre GreenCart et la liste des producteurs de `data/producteurs.json`. Sur toute autre adresse, il répond `404` avec le texte « Page introuvable ».

Ce serveur est séparé de l'API. Il tourne sur le port 3001.

### Rappel

- Le cycle d'une requête HTTP, slide 17.
- Les codes de statut, slide 20.
- Cours JavaScript : la boucle `for`, slide 34.

### Résultat attendu

Lancez-le dans un autre terminal :

```
node accueil-http.js
```

Ouvrez http://localhost:3001 : la page affiche

```
GreenCart
Des produits locaux, en direct des producteurs.
Nos producteurs
  • Ferme des Trois Chênes (Saint-Émilion)
  • Verger de Lucie (Agen)
  • Chèvrerie du Plateau (Millau)
  • Rucher Martin (Bazas)
```

http://localhost:3001/contact répond `404` « Page introuvable ».

### Indice

La fonction donnée à `http.createServer` est appelée à chaque requête. C'est à vous de regarder `req.method` et `req.url` avec un `if`. Puis :

- `res.writeHead(200, { 'Content-Type': 'text/html; charset=utf-8' })` fixe le code et dit au navigateur que c'est du HTML. Le `charset=utf-8` garde les accents.
- `res.end(page)` envoie la page et termine la réponse. Sans `res.end`, le navigateur attend sans fin.

Les étapes sont détaillées en commentaire dans le fichier.

Pour le vérificateur, laissez l'API et la page d'accueil tourner toutes les deux, puis lancez `node verifier.js` dans un troisième terminal.

### À rendre

`accueil-http.js`.

---

## Ce que vous rendez

Le dossier complet du projet, **sans** `node_modules` (il se réinstalle avec `npm install`). Ajoutez un fichier `verification.txt` avec le dernier résultat de `node verifier.js`, copié depuis le terminal.

Un exercice « À revoir » n'est pas un échec. Rendez ce que vous avez, et notez en haut de `verification.txt` ce qui vous a bloqué.